Projeto Detalhado de Software

Padrões de Projetos
(GoF Patterns)

Strategy, Decorator

Camilo Almendra | camilo@es.ufc.br | https://sites.google.com/a/es.ufc.br/camilo/

Problema

- **Planos de Operadoras**
  - Vários planos disponíveis, mas cada usuário possui apenas 1 plano base para cálculo de fatura mensal

```
public void calcular() {
  ...
  switch(cliente.plano()){
    case TODA_HORA:
      // faça algo
    case NOTURNO:
      // faça algo
    case FINAL_SEMANA:
      // faça algo
  }
  ...
}
```

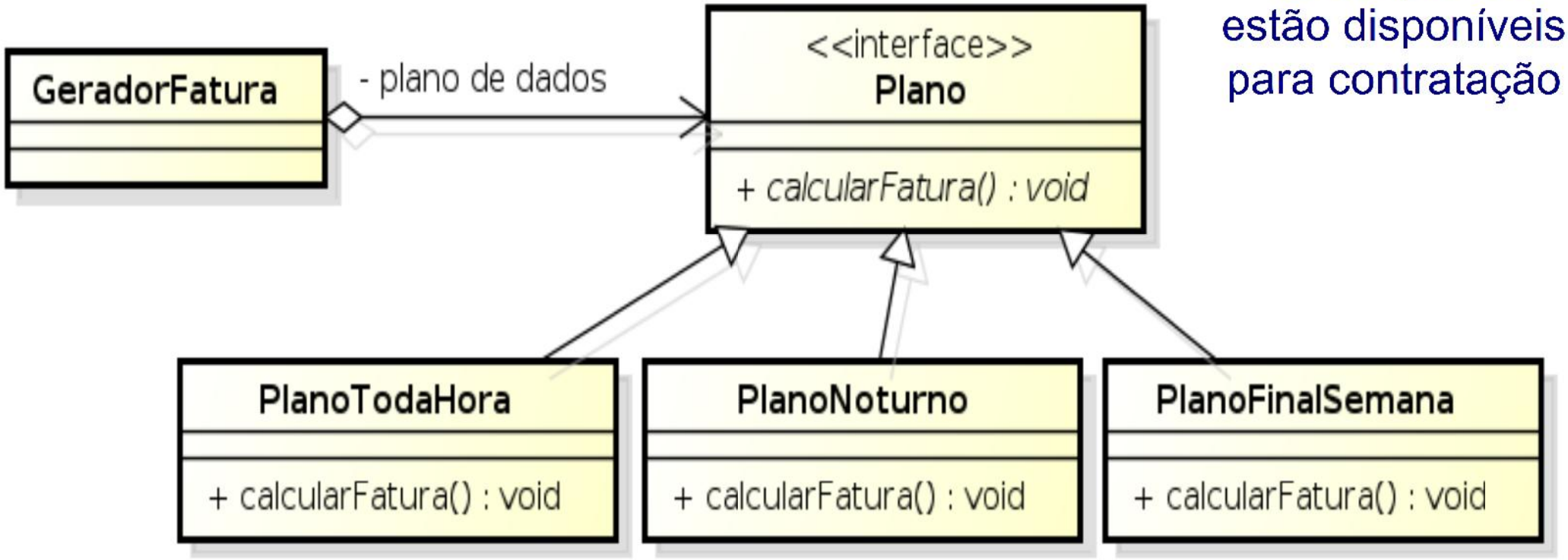


Padrão – Strategy

- **Intenção:** Definir uma família de algoritmos, encapsular cada um, fazê-los intercambiáveis.

Motivação

Vários planos estão disponíveis para contratação



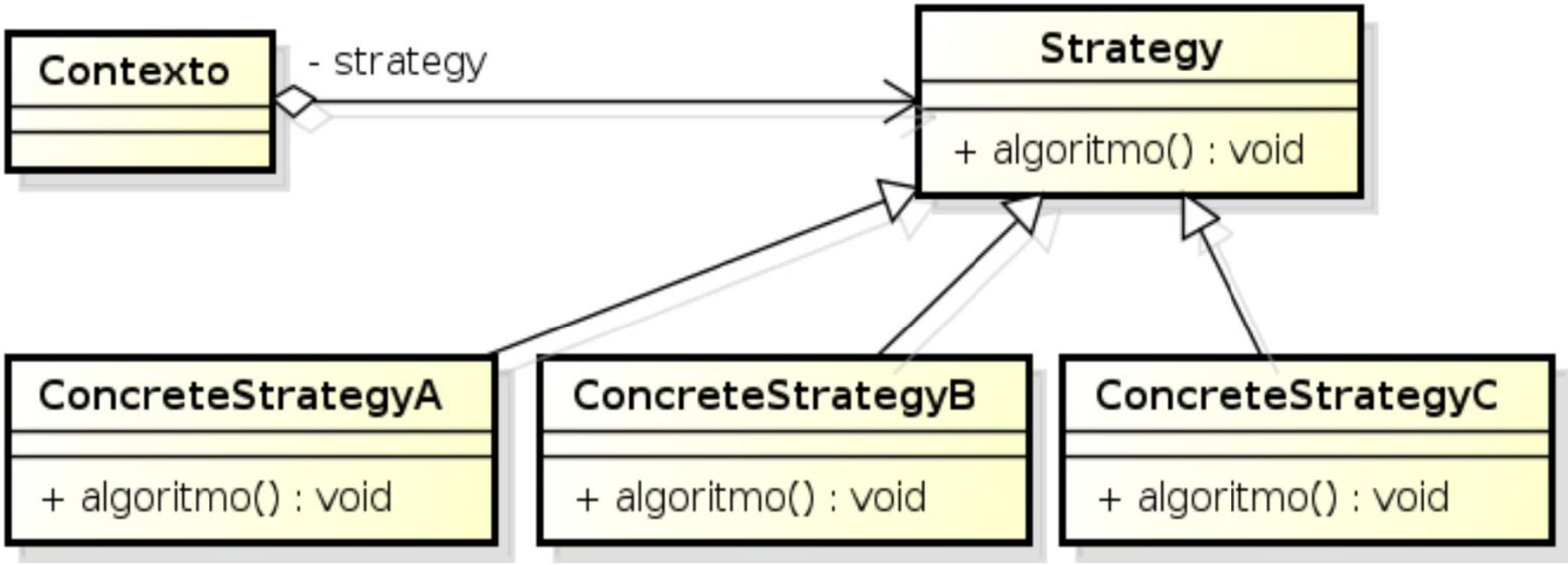
Padrão – Strategy

```
public class GeradorFatura{
  private Plano plano; //interface
  public GeradorFatura(Plano plano){
    this.plano=plano;
  }
  public void calcularFatura(){
    this.plano.calcularFatura();
  }
}
```

```
//utilizando no código
Plano plano = new PlanoNoturno();
GeradorFatura gerador = new
    GeradorFatura(plano);
gerador.calcularFatura();
```

Padrão – Strategy

- **Diagrama conceitual**



Padrão – Strategy

```
//outro exemplo de: https://robsoncastilho.com.br/2011/06/25/conhecendo-design-patterns-e-o-padrao-strategy/
public class Pedido
{
    // utilizando composição: o pedido mantém uma referência (passada em seu construtor)
    // para a Strategy base, normalmente uma interface ou classe abstrata
     private ICalculadorDeFrete _calculadorDeFrete;
    public Pedido(ICalculadorDeFrete calculadorDeFrete)
    {
        _calculadorDeFrete = calculadorDeFrete;
    }
     public decimal CalcularFrete()
    {
        // delegando o cálculo do frete para o objeto Strategy configurado
        return _calculadorDeFrete.CalcularFretePara(this);
    }
}
```

Padrão – Strategy

```
public interface ICalculadorDeFrete
{
    decimal CalcularFretePara(Pedido pedido);
}
// frete "Padrão"
public class CalculadorDeFretePadrao :
     ICalculadorDeFrete
{
    public decimal CalcularFretePara(Pedido
     pedido)
    {
        // aqui vai o algoritmo de cálculo de frete
     "padrão"
    }
}
```

```
// frete "Expresso"
public class CalculadorDeFreteExpresso :
     ICalculadorDeFrete
{
    public decimal CalcularFretePara(Pedido
     pedido)
    {
        // aqui vai o algoritmo de cálculo de
     frete "expresso"
    }
}
```

Padrão – Strategy

```
// alguma classe que utilize o Pedido
// pedido criado com o cálculo de frete padrão
var pedido1 = new Pedido(new CalculadorDeFretePadrao());
var fretePadrao = pedido1.CalcularFrete();

// pedido criado com o cálculo de frete expresso
var pedido2 = new Pedido(new CalculadorDeFreteExpresso());
var freteExpresso = pedido2.CalcularFrete();
.....
```

Padrão – Strategy

- **Aplicabilidade (discussão)**
  - Várias classes relacionadas diferem apenas no comportamento interno
  - Necessidade de diferentes variações de algoritmos
  - Algoritmos usam dados que classes clientes não deveriam ter ciência
  - Múltiplos condicionais usados em uma classe para garantir comportamentos diferentes

Padrão – Strategy

- **Participantes**
  - *Strategy* é a interface comum aos algoritmos suportados
  - *Concrete Strategy* implementa a interface *Strategy* para cada algoritmo
  - *Contexto*
    - é configurado com um objeto ConcreteStrategy
    - mantém referência a interface Strategy (polimorfismo)
    - opcionalmente pode manter um interface para acesso aos dados pela *Strategy*
- **Conseqüência**
  - Família de algoritmos relacionados
  - Evita uso de herança
  - Eliminação de condicionais (**if-else**)

Padrão – Strategy

- Exercício da Caelum
  - https://github.com/caelum/agile-workshop-tdd
- Fazer exercício 1
  - Incluir mais um tipo de cargo (ex.: Gerente)
  - Com uma regra própria de cálculo de impostos (ex.

    Pagar 25% até 5.000,00 e depois 35%)
- Como adicionar isso rápido ao código?
- Como usar o Strategy nessa situação?

Padrão – Strategy

```
public class CalculadoraDeSalario{
  ...
  public double calcula(Funcionario funcionario){
    IPercentual percentual = new
      PercentualFabrica(funcionario);
    return percentual.calcula(Funcionario);
  }
}

public interface IPercentual{
  public double calcular(Funcionario);
}
```

```
public class DesenvolvedorPercentual {
  ...
  public double calcula(Funcionario funcionario){
    if(funcionario.getSalarioBase() > 3000.0) {
            return funcionario.getSalarioBase() * 0.8;
    } else {
            return funcionario.getSalarioBase() * 0.9;
    }
  }
}
```

Padrão – Strategy

```
public class DBATESTERPercentual{
  ...
  public double calcula(Funcionario funcionario){
    if(funcionario.getSalarioBase() > 2000.0) {
            return funcionario.getSalarioBase() *
      0.75;
    } else {
            return funcionario.getSalarioBase() *
      0.85;
    }
  }
}
```

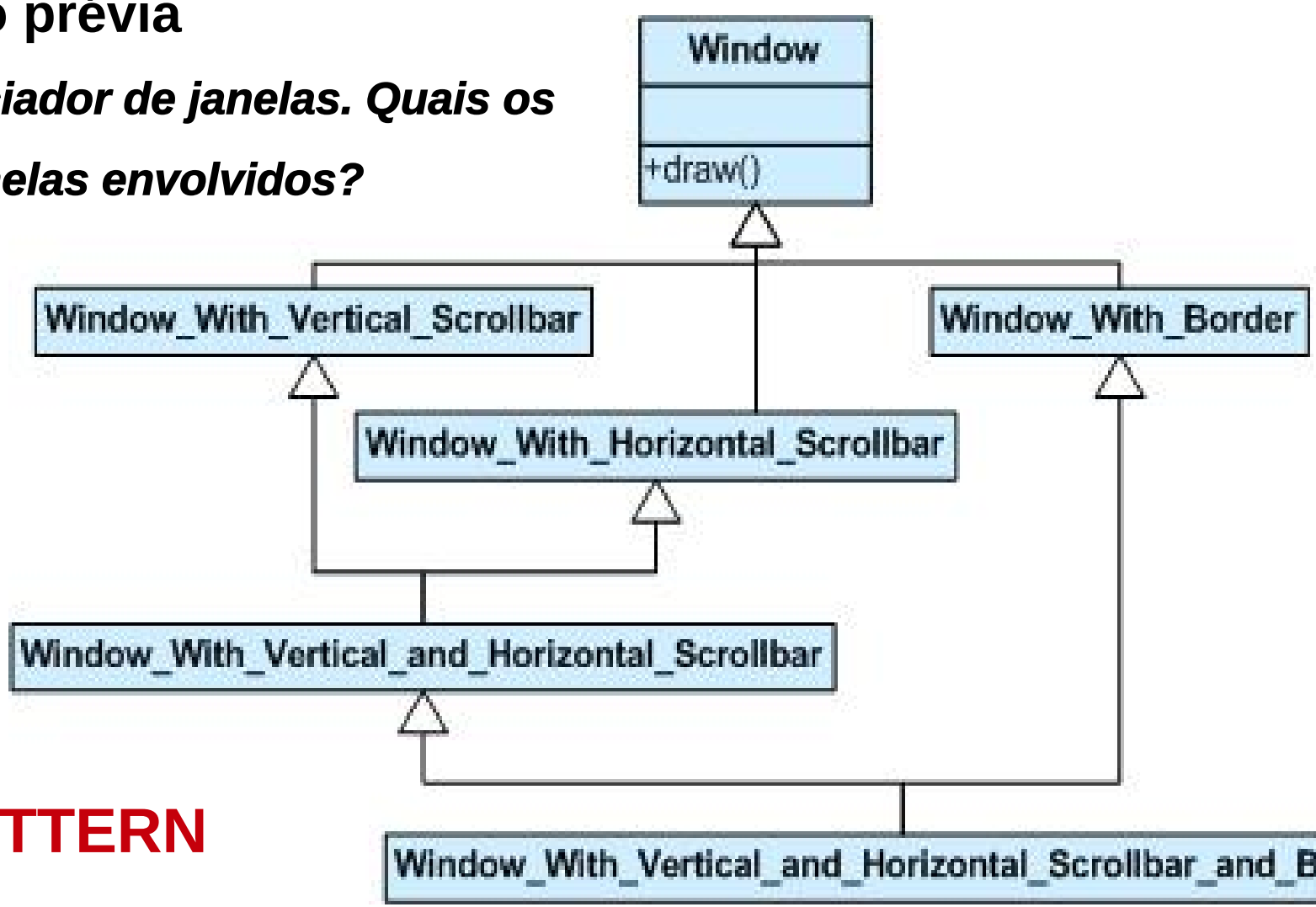
```
//misturando com fábrica
public class PercentualFabrica{
  ...
  public IPercentual criaPercentual(Funcionario
     funcionario){

     if(DESENVOLVEDOR.equals(funcionario.get
     Cargo()){
     return new DesenvolvedorPercentual();
    } else if (DBA.equals(funcionario.getCargo()) ||
     TESTER.equals(funcionario.getCargo())) {
     return new DBATESTERPercentual();
    }
  }
}
```

Padrões – Decorator

- **Discussão prévia**

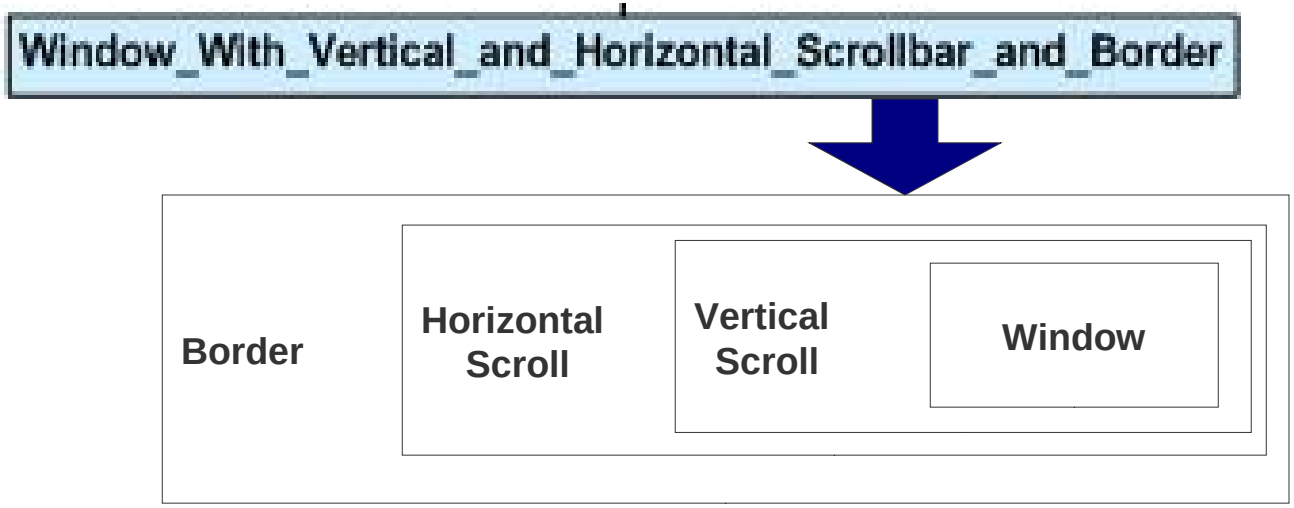
Em um gerenciador de janelas. Quais os tipos de janelas envolvidos?



ANTI-PATTERN

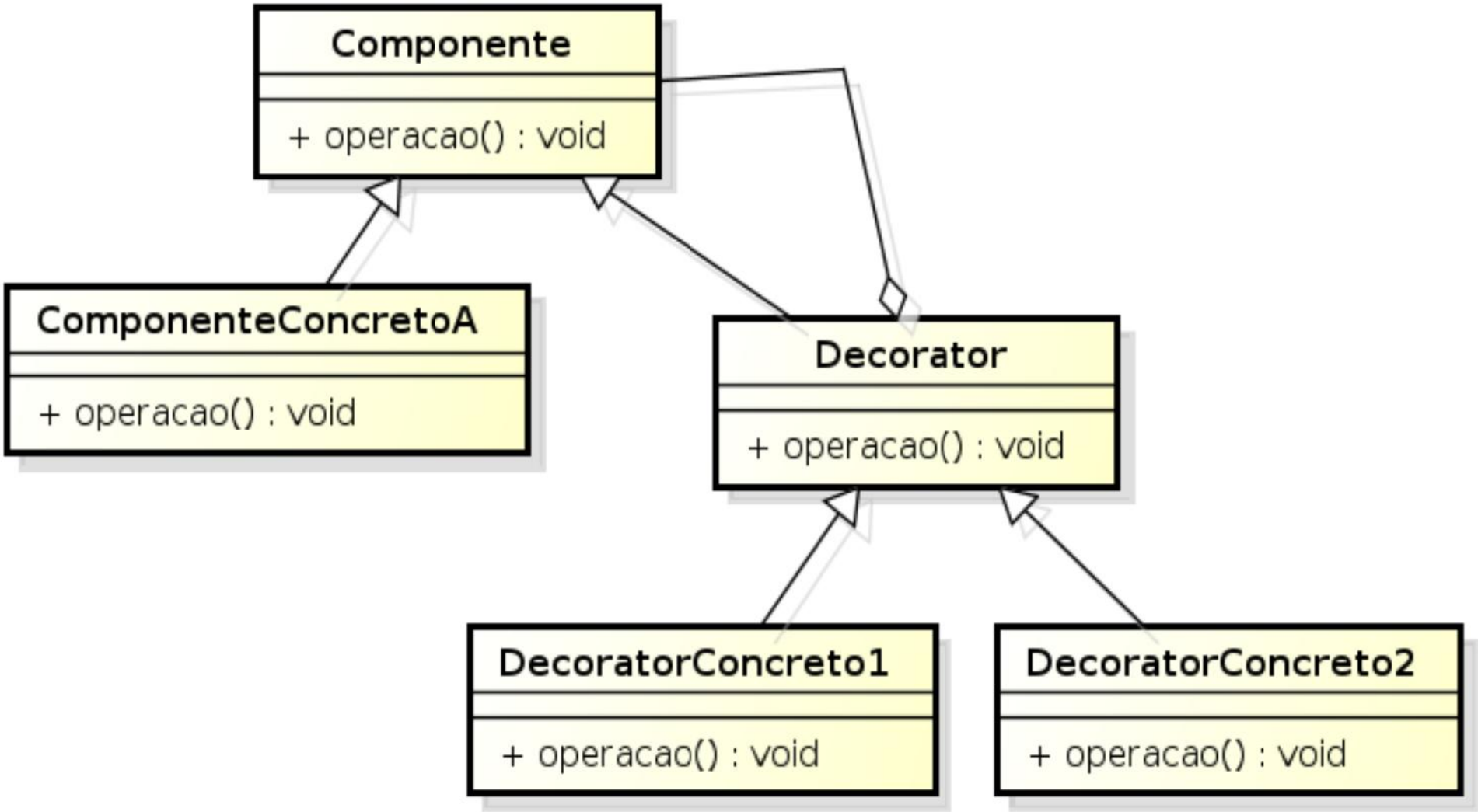
Padrão – Decorator

- **Intenção:** Agregar novas responsabilidades de forma dinâmica a um OBJETO.
- **Motivação:** Necessidade de agregar responsabilidades em tempo de execução. Apenas herança não funciona pois é estática e aplica-se a classe inteira.



Padrão – Decorator

- Diagrama Conceitual



Padrão – Decorator

- **Participantes**
  - *Componente* é a interface (classe abstrata) para objetos que podem ter responsabilidades agregada dinamicamente.
  - *ComponenteConcreto* define um objeto para o qual as responsabilidades adicionais são agregadas
  - *Decorator* mantém uma referência ao objeto Componente e define uma interface que está em conformidade com a componente.
  - *Decorators concretos* estende a funcionalidade do *Componente*.

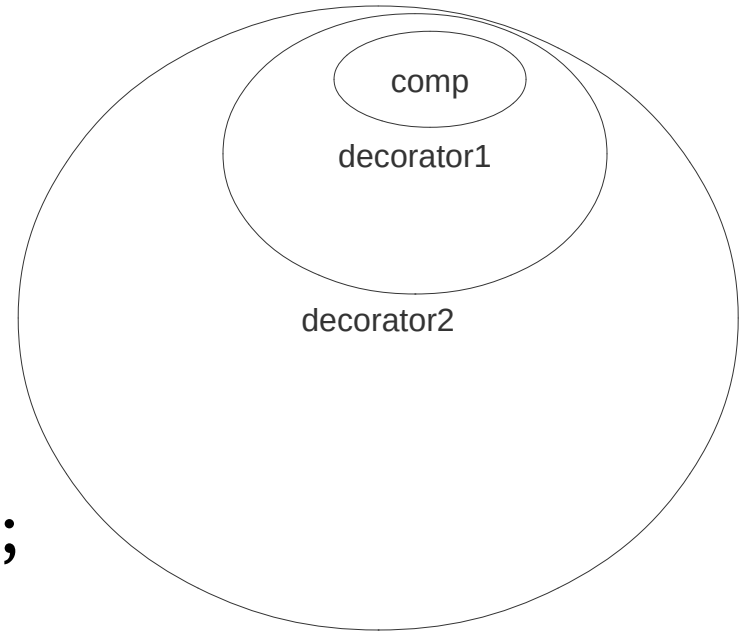
Padrão – Decorator

- Dicas:
  - Os “decorators” tem o mesmo supertipo que os objetos que eles decoram
  - Você pode usar um ou mais decorators para um objeto
  - O decorator adiciona seu próprio comportamento antes e/ou depois de delegar para o objeto que ele decora o resto do trabalho (recursão)
  - É possível usar decorators para estender comportamento em tempo de execução

Padrão – Decorator

- **Código conceitual**

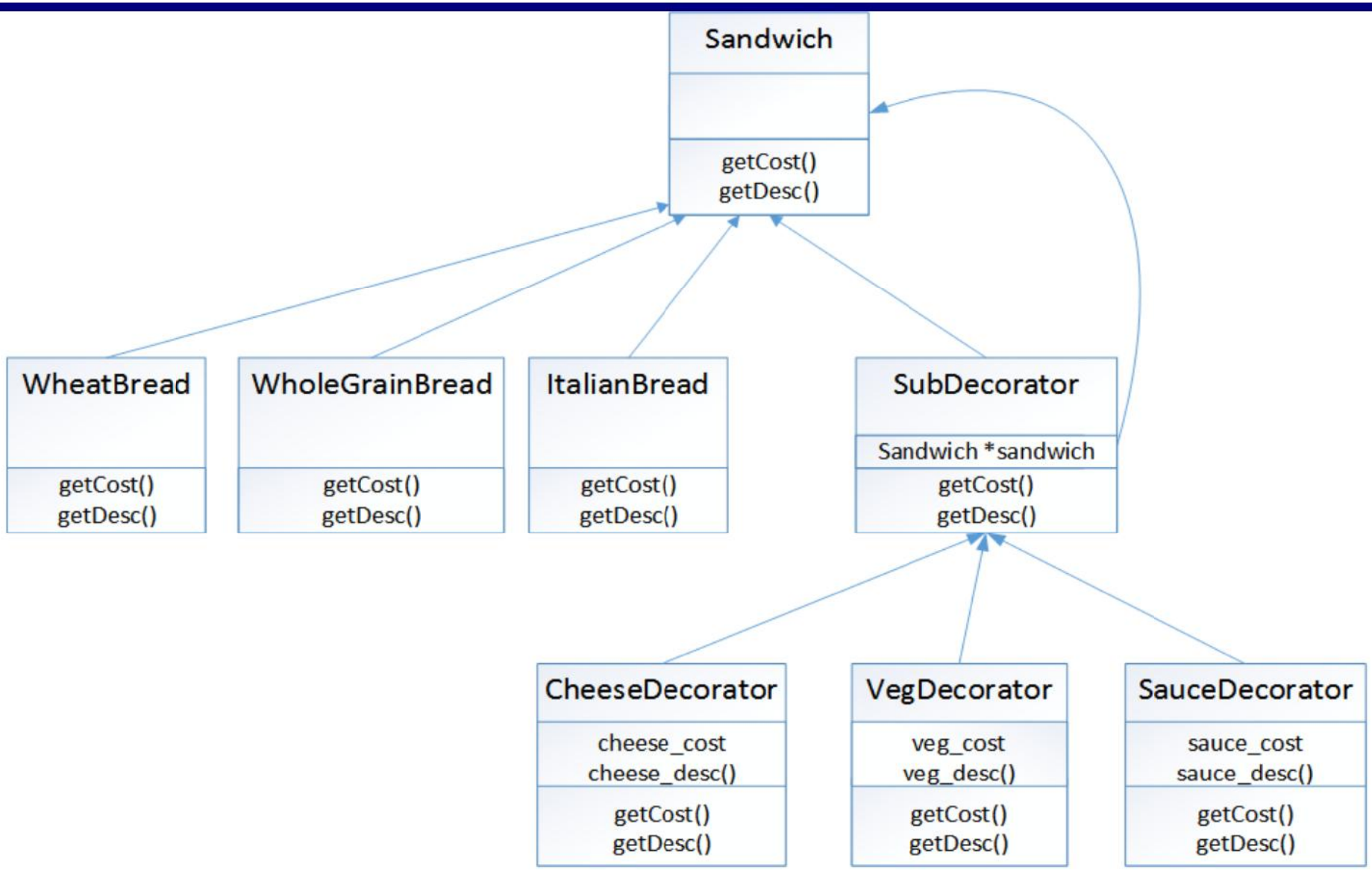
```
// Montagem dos decoradores
Componente comp =
        new ConcreteComponente();
DecoratorConcreto decorator1 =
        new DecoratorConcreto1(comp);
DecoratorConcreto decorator2 =
        new DecoratorConcreto1(decorator1);
```



Padrão – Decorator

- Exemplo: http://blog.valicode.co.uk/?page_id=65
- Exemplo Lúdico:
  - Opções de acréscimo de ingredientes em sanduíches
  - “Queijo”, “Ovo”, “Bacon”, “Pão Batata”

Padrão – Decorator



Conclusão

- Strategy

ENCAPSULA ALGORITMOS E FACILITA TROCAS ENTRE IMPLEMENTAÇÕES DE ALGORITMOS

- Decorator

PREVENI EXPLOSÃO DE SUBCLASSES E PERMITIR AGREGAR FUNCIONALIDADES OPCIONAIS EM TEMPO REAL